

REPORTE DE PRÁCTICA: TIPOS DE DATOS Y MEMORIA

Asignatura:	Programación Estructurada
Estudiante:	Aldo Ezequiel Rodriguez Godoy
Código:	224097617
Sección:	D04
Profesor:	JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha:	30 de enero del 2026

INTRODUCCIÓN

En esta práctica nos enfocamos en trabajar con los tipos de datos primitivos en el lenguaje C. Cuando programamos, a veces pensamos que las variables son solo cajas donde guardamos números o letras, pero en realidad, cada vez que declaramos una variable, le estamos pidiendo al compilador que reserve un espacio físico de bytes dentro de la memoria RAM del equipo. El tamaño de este espacio depende directamente del tipo de dato que elijamos y de la arquitectura del procesador que estemos usando (que normalmente es de 32 o 64 bits).

Por ejemplo, un `char` es el tipo más pequeño porque solo usa 1 byte para guardar un carácter en formato ASCII, mientras que los tipos flotantes como `float` o `double` usan estructuras más complejas bajo estándares de precisión matemática (como el IEEE 754) para poder almacenar números con puntos decimales.

Lo más interesante de este tema, y lo que ponemos a prueba en la actividad, es que en C podemos romper la abstracción y ver exactamente en qué parte de la memoria se guardan las cosas. Para lograr esto, utilizamos el operador de dirección `&`. Este operador nos permite extraer la dirección física en formato hexadecimal de la celda de memoria donde empieza nuestra variable. Para poder imprimir estas direcciones en la pantalla de la consola, la función `printf` tiene un formateador especial que es `%s` para texto o `%d` para enteros, pero para las direcciones de memoria usamos específicamente `%p`, el cual nos pide como buena práctica hacer una conversión o cast a un apuntador genérico (`void*`) para evitar alertas o errores al compilar.

OBJETIVO

Objetivo General

El objetivo de esta actividad es construir y probar un programa en lenguaje C que declare e inicialice los diferentes tipos de datos comunes del lenguaje, mostrando en la consola tanto el valor que tienen guardado como la dirección real de la memoria RAM donde el sistema operativo los almacenó.

Objetivos Particulares

- Declarar variables utilizando los diferentes modificadores de tamaño y signo como `short`, `long`, `unsigned` y `bool`.

- Aprender a usar los modificadores de espacio en el printf para que los datos impresos en la consola salgan alineados en forma de tabla limpia.
- Utilizar de forma correcta el operador & junto con el formato %p para mapear de manera visual cómo se acomodan las variables en la memoria durante la ejecución del programa.

DESARROLLO

Para realizar esta actividad, lo primero que hice fue crear el archivo principal del código fuente (main.c) dentro de la carpeta de código asignada. El desarrollo del algoritmo lo dividí en tres pasos principales para asegurarme de que funcionara correctamente:

1. Declaración de las variables

Empecé declarando variables para cubrir todos los tipos de datos solicitados. Declaré un char para una letra, un int normal, un unsigned int para un número que solo sea positivo, y variables más grandes usando long y long long. También incluí variables para decimales (float, double y long double). Para poder usar el tipo de dato bool (booleano), tuve que recordar incluir la librería <stdbool.h> al principio del código, y finalmente añadí un arreglo de caracteres char texto[] para meter una cadena de texto completa. A cada una le asigné un valor inicial directamente en el código para tener datos con qué probar.

2. Diseño de la tabla en consola

Uno de los puntos que requirió más cuidado fue lograr que la salida no se viera desordenada. En lugar de imprimir los datos uno abajo del otro sin formato, utilicé modificadores de ancho dentro del printf. Coloqué instrucciones como %-15s y %-25s. El signo de menos (-) sirve para alinear el texto a la izquierda, y los números indican cuántos caracteres de espacio obligatorio va a ocupar esa columna. Con esto logré crear títulos fijos para que los datos se acomoden perfectamente en columnas sin importar si el nombre del tipo de dato es corto o largo.

3. Impresión de los valores y las direcciones de memoria

Para la última columna de la tabla, que es la de las direcciones de memoria, apliqué la sintaxis de punteros. En el printf, al momento de pasar los argumentos, a cada variable le antepuse el símbolo & para extraer su ubicación, y lo puse entre paréntesis con un (void*) atrás. Esto lo hice porque el compilador de C es muy estricto y, si no transformamos la dirección a un tipo de apuntador general (vacío), pueden salir advertencias (warnings) al momento de compilar el programa con gcc.

El código que diseñé e implementé quedó de la siguiente manera:

```

#include <stdio.h>
#include <stdbool.h>

int main() {
    // 1. Declaracion e inicializacion de todos los tipos de datos comunes
    char letra = 'C';
    int entero = 100;
    unsigned int positivo = 45000;
    short small = 10;
    long grande = 1000000L;
    long long gigante = 9223372036854775807LL;
    float decimal = 5.5f;
    double preciso = 9.9999;
    long double ultra_preciso = 3.141592653589793238L;
    bool booleano = true;
    char texto[] = "Hola";

    // 2. Impresión de la cabecera de la tabla bien alineada
    printf("%-15s %-25s %-s\n", "TIPO DE DATO", "VALOR GUARDADO", "DIRECCION EN MEMORIA");
    printf("-----\n");

    // 3. Impresión de cada tipo, su valor y su direccion física usando (void*)&
    printf("%-15s %-25c %p\n", "char", letra, (void*)&letra);
    printf("%-15s %-25d %p\n", "int", entero, (void*)&entero);
    printf("%-15s %-25u %p\n", "unsigned int", positivo, (void*)&positivo);
    printf("%-15s %-25hd %p\n", "short", small, (void*)&small);
    printf("%-15s %-25ld %p\n", "long", grande, (void*)&grande);
    printf("%-15s %-25lld %p\n", "long long", gigante, (void*)&gigante);
    printf("%-15s %-25.1f %p\n", "float", decimal, (void*)&decimal);
    printf("%-15s %-25.4f %p\n", "double", preciso, (void*)&preciso);
    printf("%-15s %-25.10Lf %p\n", "long double", ultra_preciso, (void*)&ultra_preciso);

    // Para el booleano muestro un texto equivalente según su valor (1 o 0)
    printf("%-15s %-25s %p\n", "bool", booleano ? "true (1)" : "false (0)",
(void*)&booleano);

    // Para el arreglo de texto, muestro la cadena completa
    printf("%-15s %-25s %p\n", "char[] (string)", texto, (void*)&texto);

    return 0;
}

```

CONCLUSIÓN

Al principio del curso se me hacía un poco enredado entender la diferencia entre tantos tipos de datos enteros o flotantes, y no entendía bien por qué importaba tanto elegir uno u otro. Sin embargo, gracias al desarrollo de esta práctica, pude comprender que cada elección afecta directamente la memoria del equipo.

Lo que más me llamó la atención de los resultados del programa fue ver las direcciones impresas en formato hexadecimal. Si uno analiza con atención los números que arroja la consola, se puede ver cómo cambian los últimos dígitos de las direcciones. Esto me demostró de forma real cómo el compilador va acomodando una variable junto a la otra de forma consecutiva dentro del espacio de la memoria (en la pila o stack). Creo que entender este tema de cómo se distribuyen los bytes y cómo

localizar una variable con su dirección es sumamente importante, ya que nos da las bases necesarias para aprender los temas más difíciles que siguen en la materia, como lo son el uso de los apuntadores, los arreglos y la asignación dinámica de memoria.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer Systems: A Programmer's Perspective* (3rd ed.). Pearson.